

BinderDex

In-Depth Technical Project Plan

Pokémon TCG Collection Management & AI-Powered Binder Platform

Version 3.0 | December 2025

Attribute	Details
Status	Awaiting Approval
Development Model	Solo developer + Claude Code
Timeline	19 weeks (Phase 0-7)
Primary Stack	Next.js 14, Supabase, TypeScript, Magic UI, Stripe

1. Project Overview & Vision

1.1 Executive Summary

BinderDex is a web application for Pokémon TCG collectors to organize, track, and visualize their collections. The platform differentiates itself through ChromaDex, an AI-powered feature that generates aesthetically cohesive binder page layouts based on color analysis and thematic tagging.

1.2 Problem Statement

Current Pokémon TCG collection tools lack: (1) Visual binder representation mirroring physical collections, (2) Intelligent suggestions for aesthetic card arrangements, (3) Seamless master set tracking with customizable variant preferences, (4) Modern interfaces for browsing large card databases.

1.3 Core Value Pillars

Pillar	Description
Browse	Comprehensive card database with powerful multi-filter search (type, set, rarity, era, generation, legendary/mythical)
Track	Visual master set tracker showing exactly what user's physical binder should look like when complete
Build	Sandbox binder builder with drag-and-drop card placement from filtered card picker panel
Inspire	ChromaDex AI generates cohesive page layouts based on color harmony, themes, and user-specified filters

1.4 Target Audience

Primary: Casual collectors tracking collections, master set completionists, binder aesthetics enthusiasts (TikTok/Instagram/Reddit communities)

Secondary: Returning collectors from XY/Mega Evolution era, competitive players tracking deck cards

1.5 Success Metrics (12-Month Targets)

Metric	Target
Registered Users	25,000
Monthly Active Users	10,000
Pro Subscribers (5% conversion)	1,250
Monthly Recurring Revenue	\$6,250

2. Technical Architecture

2.1 System Architecture

JAMstack architecture with Supabase as Backend-as-a-Service. This minimizes infrastructure management while providing scalability, real-time capabilities, and built-in authentication.

2.1.1 Technology Stack

Layer	Technology	Rationale
Framework	Next.js 14	App Router, Server Components, ISR, optimal Vercel deployment
Language	TypeScript	Type safety, better DX, runtime error reduction
Styling	Tailwind + Magic UI	Utility-first CSS, pre-built animated components
State	Zustand + React Query	Client state + server state caching with optimistic updates
Database	Supabase PostgreSQL	Full SQL, RLS, real-time, generous free tier (500MB)
Auth	Supabase Auth	Built-in JWT, OAuth providers, guest mode support
Storage	Supabase Storage	User uploads, generated exports, 1GB free
Hosting	Vercel	Edge network, automatic HTTPS, preview deployments
Payments	Stripe	Subscriptions, webhooks, customer portal
AI	Claude API	Card tagging for ChromaDex (~\$0.003/card)

2.1.2 Why Supabase Over Alternatives?

vs Firebase: Full PostgreSQL with SQL/joins (not NoSQL), Row Level Security at DB level, standard SQL = portable

vs Planetscale: Built-in auth, storage, edge functions in one platform

vs Custom backend: Dramatically reduced development time, managed infrastructure, free tier for MVP

2.2 Key Architectural Decisions

Decision 1: Card Variants Only in Master Set Tracker

Context: Cards have variants (Normal, Reverse Holo, 1st Edition). Showing all variants in Browse would inflate card counts and confuse users.

Decision: Browse Cards shows unique cards only. Variants are stored separately and ONLY displayed in Master Set Tracker where they're relevant for completion tracking.

Implications: Separate card_variants table, different queries for Browse vs Master Set, cleaner UX.

Decision 2: Shared Filter System Across Features

Context: Browse, Binder Builder, and ChromaDex all need card filtering.

Decision: Implement ONE reusable filter system (component + hook + store) used by all three features.

Implications: Consistent UX, reduced code duplication, easier maintenance, URL-based filter sharing.

Decision 3: Pre-computed ChromaDex Data

Context: Color extraction and AI tagging are expensive operations.

Decision: Run color extraction and AI tagging as batch jobs during data import, store results in dedicated tables.

Implications: Fast ChromaDex generation at runtime, initial data processing overhead, storage for color/tag data.

3. Data Layer & Database Design

3.1 Data Source

Repository: <https://github.com/PokemonTCG/pokemon-tcg-data>

Contains JSON files for all cards/sets. We download and process locally rather than using the API to avoid rate limits and have full control.

3.1.1 Source Card Structure

Field	Type	Notes
id	string	Unique: '{setId}-{number}' (e.g., 'sv1-1', 'mee1-25')
name	string	Card name (e.g., 'Charizard ex')
supertype	string	'Pokémon', 'Trainer', or 'Energy'
subtypes	string[]	['Stage 2', 'ex'], ['Item'], ['Basic']
types	string[]	Pokémon types: ['Fire'], ['Water', 'Psychic']
rarity	string	'Common', 'Illustration Rare', 'Special Illustration Rare'
number	string	Card number (supports '001', 'TG01', etc.)
nationalPokedexNumbers	number[]	Pokédex numbers (multiple for TAG TEAM cards)
images.small/large	string	Image URLs from pokemontcg.io CDN
artist	string	Card illustrator

3.1.2 Data Import Considerations

Rarity Normalization

Source has inconsistent strings ('Rare Holo' vs 'Holo Rare'). Create mapping table to normalize to enum values.

Premium Card Identification

Mega Evolution & S/V: isPremium = true for 'Illustration Rare', 'Special Illustration Rare'

Pre-S/V: 'Rare Ultra' contains mixed content - requires manual tagging or lookup table

Legendary/Mythical Classification

NOT in source data. Create static lookup of legendary/mythical Pokédex numbers (Articuno=144, Mewtwo=150, etc.) and cross-reference during import.

Generation Derivation

Map nationalPokedexNumbers to generations: 1-151=Gen1, 152-251=Gen2, etc. For multi-Pokémon cards, use primary Pokémon's generation.

3.2 Database Schema Overview

3.2.1 Core Tables

sets: id (PK), name, series, era (enum), printed_total, total, release_date, has_reverse_holos, logo_url, symbol_url

cards: id (PK), set_id (FK), name, number, rarity (enum), supertype (enum), subtypes[], types[], hp, artist, national_dex_numbers[], image_small, image_large, is_promo, is_premium, is_legendary, is_mythical, generation

card_variants: id (PK), card_id (FK), variant_type (enum: NORMAL, REVERSE_HOLO, FIRST_EDITION, etc.), image_url

3.2.2 ChromaDex Tables

card_colors: id (PK), card_id (FK UNIQUE), dominant_hex, dominant_hsl (JSONB), palette (JSONB), brightness, saturation, warmth

card_tags: id (PK), card_id (FK), tag, category (enum: THEME, SETTING, MOOD, COMPOSITION, SUBJECT), confidence

3.2.3 User Tables

user_profiles: id (PK = auth.users.id), email, display_name, avatar_url, tier (enum), chromadex_uses_this_month, chromadex_reset_date

user_collections: id (PK), user_id (FK), variant_id (FK), quantity, condition (enum), notes, acquired_date

subscriptions: id (PK), user_id (FK), stripe_customer_id, stripe_sub_id, status (enum), current_period_end

3.2.4 Binder Tables

binders: id (PK), user_id (FK), name, description, slot_config (enum: NINE/TWELVE/SIXTEEN), type (enum: CUSTOM/MASTER_SET/CHROMADEX), master_set_id (FK), is_public, is_curated

binder_pages: id (PK), binder_id (FK), page_number

binder_slots: id (PK), page_id (FK), position (0-8/0-11/0-15), variant_id (FK nullable)

master_set_preferences: id (PK), user_id (FK), set_id (FK), slot_config, include_promos, include_reverse_holos

3.3 Variant System Deep Dive

Not all cards have all variant types. Rules vary by era:

Era	Available Variants
Base Set	NORMAL, FIRST_EDITION, SHADOWLESS
Jungle - Neo	NORMAL, FIRST_EDITION
e-Card through B&W	NORMAL, REVERSE_HOLO
XY through Sun & Moon	NORMAL, REVERSE_HOLO (most non-holo cards)
Sword & Shield	NORMAL, REVERSE_HOLO (most cards)
Scarlet & Violet	NORMAL, REVERSE_HOLO (Commons/Uncommons/Rares only, NOT Double Rare+)
Mega Evolution (2025)	NORMAL, REVERSE_HOLO (TBD - research needed)

Critical: During data import, we must correctly populate card_variants based on era-specific rules. Incorrect variant data breaks Master Set Tracker accuracy.

3.4 Database Indexes

Critical for query performance:

cards: set_id, rarity, types (GIN), is_premium, isLegendary, isMythical, generation, name (full-text)

card_colors: dominant_hsl (consider GiST for range queries)

card_tags: tag, category

user_collections: user_id + variant_id (unique), user_id

binders: user_id, is_public

3.5 Row Level Security (RLS)

Public read: cards, sets, card_variants, card_colors, card_tags

User-owned: user_profiles, user_collections, binders (own or public), subscriptions

RLS policies ensure users can only access their own data. Public binders readable by all.

4. Feature Specifications

4.1 Shared Filter System

Critical: Same filter system used in Browse Cards, Binder Builder Card Picker, and ChromaDex constraints.

4.1.1 Filter Categories

Filter	Type	Implementation
Text Search	String	ILIKE or full-text search on name, artist
Set	Multi-select	IN clause on set_id
Era	Multi-select	Join to sets.era
Type	Multi-select	PostgreSQL array @> operator
Rarity	Multi-select	IN clause, grouped by tier in UI
Generation	Dropdown 1-9	WHERE generation = ?
Supertype	Radio buttons	Pokémon/Trainer/Energy
Premium	Toggle	WHERE is_premium = true
Legendary	Toggle	WHERE is_legendary = true
Mythical	Toggle	WHERE is_mythical = true

4.1.2 State Management

Zustand store: Holds current filter state, provides actions for updates

URL sync: Filters serialized to URL params for shareability (?types=Fire&era=MEGA_EVOLUTION)

React Query: Fetches data with filter dependencies, handles caching

Debounce: 300ms on text search to reduce API calls

4.2 Card Database Browser

4.2.1 Core Requirements

- Display UNIQUE CARDS ONLY (no variants) - prevents inflated counts
- Responsive grid: 4 columns (desktop), 3 (tablet), 2 (mobile)
- Infinite scroll with virtualization for 1000+ cards
- Shimmer loading states, hover effects with quick info
- Server-side pagination (50 cards/page default)

4.2.2 Card Detail Modal

- Full card image with zoom capability
- All metadata: HP, types, artist, set, number, rarity
- Variant list (informational): 'This card exists as: Normal, Reverse Holo'
- Actions: Add to Collection, Add to Binder, Use in ChromaDex

4.3 Master Set Tracker

4.3.1 Visual Binder Concept

Key insight: NOT a checklist - it's a visual representation of what the user's physical binder should look like.

- Displays cards in binder page format (3×3, 3×4, or 4×4)
- Owned cards show full image
- Missing cards show greyed placeholder with name/number
- Click missing card → opens detail to learn about it
- Page navigation to browse full binder

4.3.2 Dynamic Preferences

Preference	Behavior
Include Promos	Toggle whether promo cards from the set are included
Include Reverse Holos	Toggle reverse holo variants (significantly increases card count)
Slot Configuration	9 (3×3), 12 (3×4), or 16 (4×4) - affects page layout

4.3.3 Layout Generation

Algorithm: 1) Sort by number → 2) Interleave/append RH if enabled → 3) Secret rares at end → 4) Promos at end if enabled → 5) Paginate to slot config

Deterministic: Same preferences ALWAYS produce same layout for consistency

4.4 Binder Builder

4.4.1 Split-Panel Layout

Left Panel (Card Picker): Search bar, full filter system, scrollable card grid with draggable cards, card count indicator

Right Panel (Binder Canvas): Current page view, droppable slots, page navigation, add/remove page buttons, binder settings

4.4.2 Drag and Drop

Library: @dnd-kit/core

- DndContext wraps both panels
- Cards in picker are Draggable sources
- Binder slots are Droppable targets
- DragOverlay shows card being dragged
- Visual feedback on valid drop targets
- Optimistic updates for responsiveness

4.4.3 Edge Cases

- Same card can be in multiple slots (user owns duplicates)
- Auto-save with 2-second debounce
- Handle multi-tab editing conflicts

5. ChromaDex: AI Engine Specifications

5.1 Overview

ChromaDex generates aesthetically cohesive binder page suggestions based on color analysis, thematic tagging, and user-specified filters. This is the flagship differentiating feature.

5.2 Pre-Processing Pipelines

5.2.1 Color Extraction Pipeline

When: Batch job during data import, incremental for new cards

Tool: node-vibrant or colorthief

Process: Download image → Extract dominant + palette (5 colors) → Convert to HSL → Calculate brightness/saturation/warmth → Store in card_colors

Scope: Process premium cards first (higher quality, most relevant for ChromaDex)

5.2.2 AI Tagging Pipeline

When: Batch job for premium cards

Tool: Anthropic Claude API (Sonnet for cost efficiency)

Process: Send image to Claude Vision → Request tags from taxonomy → Store tags with confidence > 0.7

Tag Taxonomy

- TIME: sunrise, day, sunset, dusk, night, starry
- WEATHER: sunny, cloudy, rainy, stormy, snowy, foggy
- SETTING: forest, ocean, mountain, city, sky, cave, desert, meadow
- MOOD: peaceful, energetic, mysterious, playful, dramatic, serene
- ACTION: battle, resting, running, flying, eating, playing
- STYLE: portrait, full_body, action_pose, landscape, close_up

Cost: ~\$0.003/card (Sonnet). 1,000 premium cards = ~\$3

5.3 Generation Algorithm

5.3.1 Generation Modes

Mode	Description
Color - Dominant	Match cards with similar dominant color to user's target color
Color - Palette	Match cards that fit within a color gradient
Color - Harmony	Use color wheel rules (complementary, analogous, triadic, split-complementary)
Theme	Match cards by thematic tags (e.g., 'sunset + forest + peaceful')
Hybrid	Combine color + theme constraints

5.3.2 Filter Integration

Critical: ChromaDex uses the SAME filter system as Browse Cards

Examples:

- 'Blue page with Water types from Gen 1, Premium cards only'
- 'Sunset forest theme from Scarlet & Violet'
- 'Pink Fairy legends'

5.3.3 Scoring Algorithm

Color Scoring: CIEDE2000 color difference in LAB space. Score = $1 - (\text{distance} / \text{max_distance})$

Theme Scoring: $\text{sum}(\text{matching_tag_confidence}) / \text{target_theme_count}$

Diversity Bonus: -0.3 penalty if same Pokémon already selected

5.3.4 Generation Flow

- 1) Apply all user filters → candidate pool
- 2) Validate pool size $\geq$ slot count (else error with suggestion)
- 3) Score all candidates by color/theme criteria
- 4) Select top cards with diversity bonus
- 5) Arrange in grid (highest score → center focal point)
- 6) Calculate coherence score (0-100%)
- 7) Return preview with explanation

5.4 Usage Limits

Tier	Limit	Reset
Guest	0 (requires account)	N/A
Free	3 generations/month	Monthly
Pro	Unlimited	N/A

6. Export & Import

6.1 Export Functionality

6.1.1 Basic Export (Free)

Format: CSV or JSON

Content: Card name, set, number, rarity, quantity, condition

Use case: Backup collection, import to spreadsheet

6.1.2 Premium PDF Export (Pro)

Format: PDF binder pages with card images

Options: Page size, orientation, include/exclude card names, cover page

Library: @react-pdf/renderer or jspdf

Use case: Print physical checklist, share visual binder

6.2 Import Functionality

6.2.1 Supported Sources

Source	Format & Parsing Strategy
TCGPlayer	CSV export. Match by name + set + number. Handle edge cases (misspellings, variant names)
Collectr	JSON export. Direct field mapping where possible
Custom CSV	Column mapping UI for user-defined CSVs

6.2.2 Import Flow

- 1) Upload file → Parse → Show match results
- 2) Matched cards: auto-import
- 3) Ambiguous matches: user reviews and selects correct card
- 4) Unmatched cards: show list for manual resolution
- 5) Confirm → Add to collection

7. Security & Authentication

7.1 Authentication

Providers: Email/password (primary), Google OAuth, Discord OAuth

Guest mode: Anonymous sessions with upgrade path to full account

Session: JWT in HTTP-only cookies. Access: 1hr, Refresh: 7 days

Upgrade flow: Guest data migrates to new account on sign-up

7.2 Authorization

RLS Policies: Enforced at database level via Supabase

- Cards/Sets/Colors/Tags: Public read
- User data: `auth.uid() = user_id`
- Binders: Own OR `is_public = true`

7.3 API Security

- All routes verify Supabase session
- Rate limiting: 100 req/min (free tier)
- Input validation: Zod schemas
- CSRF: SameSite cookies

8. Performance Optimization

8.1 Frontend

Bundle: Target < 200KB initial JS. Dynamic imports for ChromaDex, PDF export

Images: next/image for WebP, responsive sizes, blur placeholders, lazy loading

Lists: @tanstack/react-virtual for 1000+ card grids

8.2 Backend

Queries: Select needed columns only, use indexes, paginate (LIMIT/OFFSET)

Caching: React Query with staleTime. Cards: 24hr, User data: 5min

8.3 Targets

Metric	Target	Tool
First Contentful Paint	< 1.5s	Lighthouse
Largest Contentful Paint	< 2.5s	Lighthouse
Time to Interactive	< 3.5s	Lighthouse
Card grid load (50)	< 500ms	Network
ChromaDex generation	< 3s	API timing

9. Monetization

9.1 Tier Comparison

Feature	Guest	Free	Pro \$4.99/mo
Browse Cards	✓	✓	✓
Master Set Trackers	1	1	Unlimited
Custom Binders	1 (10 pages)	1 (10 pages)	Unlimited
ChromaDex	✗	3/month	Unlimited
Basic Export (CSV)	✗	✓	✓
Premium Export (PDF)	✗	✗	✓
Import	✗	✓	✓
Ads	Yes	Yes	No

9.2 Stripe Implementation

Products: Pro Monthly (\$4.99), Pro Yearly (\$39.99 - 33% discount)

Flow: Upgrade click → Stripe Checkout → Webhook → Update user tier

Webhooks: checkout.session.completed, customer.subscription.updated/deleted, invoice.payment_failed

Customer Portal: Self-service subscription management

10. Deployment & DevOps

10.1 Environments

Environment	URL	Purpose
Development	localhost:3000	Local dev
Preview	[branch].vercel.app	PR testing
Production	binderdex.com	Live app

10.2 CI/CD

GitHub Actions: On push/PR → Lint → Type check → Test → Build → Deploy

Preview deployments: Automatic on PRs via Vercel

Database migrations: Prisma migrations run in CI before deploy

10.3 Environment Variables

NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY,
SUPABASE_SERVICE_ROLE_KEY, STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET,
ANTHROPIC_API_KEY, DATABASE_URL

11. Development Phases

11.1 Phase Overview

Phase	Weeks	Deliverables
0	1-2	Foundation: Next.js, Supabase, Prisma, Auth, CI/CD, Data import pipeline
1	3-4	Card Browser: Grid view, shared filter system, search, card detail modal
2	5-7	Master Set Tracker: Visual binder, preferences, progress tracking, variant display
3	8-9	Binder Builder: Split-panel, card picker, drag-drop, page management
4	10-13	ChromaDex: Color extraction, AI tagging, generation algorithm, UI, limits
5	14-15	Export/Import: CSV export, PDF export, TCGPlayer/Collectr import
6	16-17	Community: Binder gallery, sharing, curated binders, polish, onboarding
7	18-19	Monetization & Launch: Stripe, feature gating, ads, launch prep

11.2 Initial Target Sets

Mega Evolution Base Set: First set in the new 2025 Mega Evolution series

Phantasmal Flames: Second set, released November 14, 2025. Features Mega Charizard X ex, Mega Gengar ex

These sets allow testing with current data while the app is developed.

12. Risk Assessment

Risk	Likelihood	Impact	Mitigation
Card data outdated	High	Medium	Monthly sync from GitHub, admin update tools
ChromaDex poor results	Medium	High	Extensive testing, feedback loop, algorithm tuning
Variant rules incorrect	Medium	High	Research each era thoroughly, community validation
Image loading slow	High	Medium	CDN, lazy loading, blur placeholders, image optimization
Supabase limits hit	Low	Medium	Monitor usage, upgrade plan, optimize queries
Stripe webhook failures	Low	High	Retry logic, signature verification, manual reconciliation

13. Approval Checklist

Please confirm the following before implementation:

- ☐ Supabase as sole backend (Auth + DB + Storage)
- ☐ No variants in Browse - only in Master Set Tracker
- ☐ Binder Builder split-panel layout with Card Picker
- ☐ ChromaDex uses same filter system as Browse
- ☐ Binder sharing in Phase 6 (Community)
- ☐ 19-week timeline acceptable
- ☐ Initial sets: Mega Evolution Base + Phantasmal Flames
- ☐ Premium card strategy: IR/SIR for modern, manual for older
- ☐ Data source: PokemonTCG/pokemon-tcg-data GitHub repo

13.1 Next Steps After Approval

1. Set up Supabase project and link MCP
2. Initialize Next.js project with TypeScript, Tailwind, Magic UI
3. Create database schema and run migrations
4. Build data import pipeline for Mega Evolution sets
5. Begin Phase 0 foundation work

— End of Technical Specification —